Password Policy Validation Engine

Problem Statement: Evaluate sample password strings based on minimum length, character categories, repetition, and prohibited-substring rules. Display each failed condition and determine whether the password is valid.

1. Computational Thinking

Computational thinking means solving a problem step by step using logical methods.

1. Decomposition: Divide the problem into smaller tasks:

- Read the password.
- Check minimum length.
- Check uppercase and lowercase letters.
- Check digits and special characters.
- Check repeated characters.
- Check prohibited words.
- Display the result.

2. Pattern Recognition: Identify common patterns in passwords, such as repeated characters, missing digits, and prohibited words.

3. Abstraction: Focus only on the password rules and ignore unnecessary information.

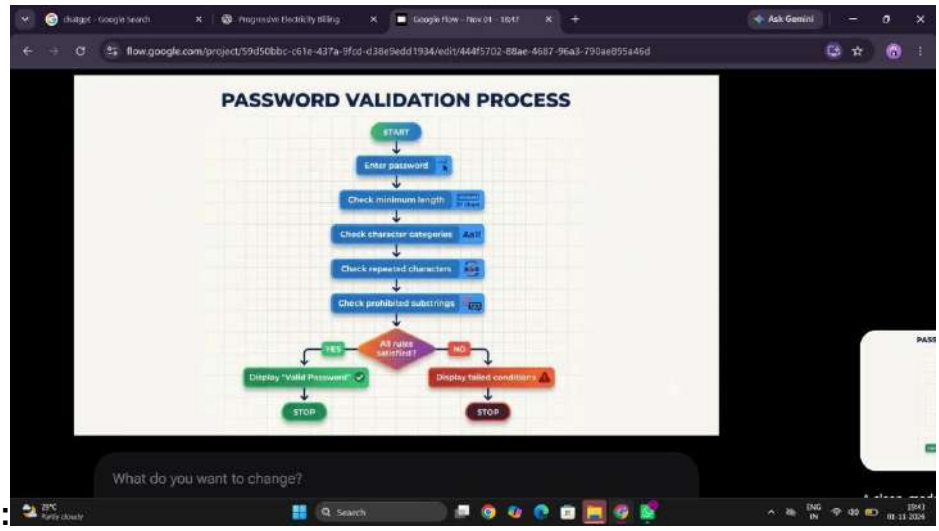
4. Algorithm Design: Create step-by-step instructions to validate the password.

2. Algorithm

1. Start.
2. Define minimum password length as 8.
3. Read the password.
4. Initialize all validation flags as False.
5. Check password length.
6. Check uppercase letters.
7. Check lowercase letters.
8. Check digits.
9. Check special characters.
10. Check consecutive repeated characters.

11. Check prohibited substrings such as password and admin.
12. Store all failed conditions in a list.
13. If the list is empty, display Password Valid.
14. Otherwise, display Password Invalid and explain the failed conditions.
15. Stop.

3. Flowchart:



4. Variables

Variable	Purpose
password	Stores input password
min_length	Minimum length (8)
has_upper	Checks uppercase letter
has_lower	Checks lowercase letter
has_digit	Checks digit
has_special	Checks special character
has_repeat	Checks repeated characters
has_prohibited	Checks prohibited words
errors	Stores failed conditions
valid	Stores final result

5. Data Types

Data type	Example
String	"Hello@123"
Integer	8
Boolean	True, False
List	["No digit", "No uppercase"]
Tuple	("password", "admin")

6. Operators Used

Operator	Purpose
>=	Checks minimum length
==	Compares values
in	Checks substring
not in	Checks absence
and	Combines conditions
or	Checks multiple conditions
not	Reverses condition

7. Conditional Statements

- if – Checks whether a condition is true.
- elif – Checks another condition.
- else – Executes when conditions are false.

Example:

```
if len(password) >= 8:    print("Length Valid")else:    print("Length Invalid")
```

8. Loops

For loop: Checks every character in the password.

```
for ch in password:    if ch.isdigit():        has_digit = True
```

While loop: Can repeatedly accept passwords until the user chooses to stop.

9. Strings

Strings are used to store passwords.

Useful string methods:

- isupper() – Checks uppercase.
- islower() – Checks lowercase.
- isdigit() – Checks digits.
- len() – Finds password length.
- lower() – Converts text into lowercase.

10. Pattern Generation

Pattern generation can be used to display validation results.

Example:

Password Validation Report

Length : PASS

Uppercase : PASS

Lowercase : PASS

Digit : PASS

Special Char : PASS

Repetition : FAIL

11. Lists

A list stores all failed conditions.

errors = []errors.append("Minimum length not satisfied")errors.append("Digit is missing")

12. Tuples and Packing

A tuple stores fixed prohibited words.

prohibited = ("password", "admin", "welcome")result = (True, False, True)

Tuple packing:

password_rules = (8, True, True, True, True)

Here, the values represent minimum length and the required character categories.

13. Pseudocode

START

```
SET min_length = 8
SET prohibited = ("password", "admin", "welcome")

INPUT password

SET errors = empty list
SET upper = False
SET lower = False
SET digit = False
SET special = False
SET repeat = False

FOR each character in password

    IF character is uppercase
        SET upper = True

    IF character is lowercase
        SET lower = True

    IF character is digit
        SET digit = True

    IF character is special character
        SET special = True

END FOR
```

```
IF length(password) < min_length
    ADD "Minimum length failed" to errors

IF upper == False
    ADD "Uppercase missing" to errors

IF lower == False
    ADD "Lowercase missing" to errors

IF digit == False
    ADD "Digit missing" to errors

IF special == False
    ADD "Special character missing" to errors

CHECK consecutive repeated characters

IF repeated characters found
    ADD "Repeated characters found" to errors

FOR each word in prohibited
    IF word is present in lowercase(password)
        ADD "Prohibited substring found" to errors

IF errors is empty
    DISPLAY "Password Valid"
ELSE
    DISPLAY "Password Invalid"
    DISPLAY all failed conditions
```

STOP

14. Sample Input and Output

Sample password	Result	Reason
Hello@123	Valid	All rules satisfied
hello123	Invalid	Uppercase and special character missing
HELLO@123	Invalid	Lowercase missing
Hi@1	Invalid	Minimum length not satisfied
Admin@123	Invalid	Prohibited substring
Hellooo@123	Invalid	Repeated characters

Assumption: Three or more consecutive identical characters are considered repetition. Prohibited words are checked without considering uppercase or lowercase differences.

15. Python Program

password = input("Enter password: ")min_length = 8prohibited = ("password", "admin", "welcome")errors = []upper = any(ch.isupper() for ch in password)lower = any(ch.islower() for ch in password)digit = any(ch.isdigit() for ch in password)special = any(not ch.isalnum() for ch in password)if len(password) < min_length: errors.append("Minimum 8 characters required")if not upper: errors.append("Uppercase letter missing")if not lower: errors.append("Lowercase letter missing")if not digit: errors.append("Digit missing")if not special: errors.append("Special character missing")# Check three consecutive repeated charactersfor i in range(len(password) - 2): if password[i] == password[i+1] == password[i+2]: errors.append("Repeated characters found") break# Check prohibited wordsfor word in prohibited: if word in password.lower(): errors.append("Prohibited substring found") breakif len(errors) == 0: print("Password Valid")else: print("Password Invalid") for error in errors: print("-", error)

16. Conclusion

The Password Policy Validation Engine checks passwords using computational thinking, algorithms, conditional statements, loops, strings, lists, tuples, and operators. It identifies invalid passwords and explains each failed condition.